Part B: Sharing One Fronthaul Link — Presenter's Report

What this is. Everything you need to know before presenting Part B of the B.Tech. project, starting from zero. It assumes no background in mobile networks, forecasting or optimisation. Read Sections 1 to 7 once from top to bottom; they build on each other. Section 8 covers the results, Section 9 the extended study, Section 10 the limitations, Section 11 a slide-by-slide speaking guide for the Part B slides of `presentation/btp_final.tex`, Section 12 the likely questions, and Section 13 a one-page cheat sheet of numbers for the last ten minutes before you go on.

Every number here comes from files in this repository (mainly `results/fhlm/*.csv`, `docs/fhlm/results_summary.md` and `docs/fhlm/extended_study.md`). Nothing has been rounded in a way that changes its meaning.

Contents

1. The background, from zero
2. The Part B problem in plain words
3. The system model with real numbers
4. Key idea 1: the deadline-feasible rate r*
5. Key idea 2: forecast a range, not a number
6. Key idea 3: the KKT (newsvendor) allocation rule
7. How the experiments were run
8. Results
9. The extended study
10. Limitations and what we do NOT claim
11. Slide-by-slide speaking guide
12. Likely questions and honest answers
13. Cheat sheet: the numbers to remember
14. Glossary
15. Where things live in the repository, and a live demo

---

1. The background, from zero

1.1 A base station is split into two boxes

When your phone talks to the network, it talks to a **base station** (the antennas on a tower or rooftop). In modern networks ("Open RAN", O-RAN) the base station is split into two physical pieces:

- The **O-RU (radio unit)** sits on the tower, next to the antennas. It does very little processing: it converts between radio waves and digital samples.
- The **O-DU (distributed unit)** sits in a nearby building or data centre. It does the heavy work: deciding which user gets which radio resources (scheduling), coding, modulation and so on.

The cable (usually fibre with Ethernet) between them is called the **fronthaul**.

```
phone  ))) [ O-RU on tower ] ===== fronthaul (Ethernet) ===== [ O-DU in a building ] --- core
```

1.2 Why the fronthaul carries so much data

How the work is split between the O-RU and the O-DU is called the **functional split**. O-RAN uses split **7-2x**: the O-RU does almost nothing, so the O-DU sends it nearly raw radio samples, called **IQ samples** (complex numbers with an "in-phase" part I and a "quadrature" part Q).

That is a lot of data. One 100 MHz cell with 4 antenna layers can need up to about **7.4 Gbit/s** of fronthaul.

1.3 Words you must know before anything else

Term	Meaning in one line
Slot	The basic time step of 5G NR. Here 0.5 ms (30 kHz subcarrier spacing). Everything in the simulator happens slot by slot.
PRB (physical resource block)	The smallest chunk of spectrum the scheduler hands out: 12 subcarriers for one slot (14 OFDM symbols). A 100 MHz cell has **273 PRBs**.
Layer	One spatial stream (MIMO). Each cell here has **4 layers**. A "PRB-layer" is one PRB on one layer.
BFP (block floating point)	The standard O-RAN way to compress IQ samples: a block of 12 numbers shares one exponent and each keeps a short **mantissa** (e.g. 9 bits). Fewer bits means less data but more noise. This is the link to Part A.
eCPRI	The protocol that carries IQ samples over Ethernet. Adds about 8 % overhead.
Spectral efficiency (η)	How many user bits fit into one resource. Good radio conditions → high η.
eMBB	"Enhanced mobile broadband": normal data (video, downloads). Tolerant of delay.
Low-latency (LL)	Traffic that must arrive fast (XR/AR, industrial control). Strict deadline.
Deadline	How long a bit may wait in the O-DU queue before it becomes useless and is thrown away.

1.4 The key fact: fronthaul load follows resource blocks, not user bits

In split 7-2x the O-DU sends IQ samples for every **scheduled PRB**, not for every user bit. So the fronthaul load of a cell is:

> fronthaul bits = (user bits ÷ spectral efficiency) × bits per PRB-layer

A cell with poor radio conditions (low η) needs more PRBs for the same user data, so it loads the fronthaul more. Two cells carrying the same user traffic can differ by 2 to 3 times in fronthaul load. In our default configuration η ranges from 2.0 to 5.5 bits per resource element across the 8 cells.

1.5 Why several cells share one link, and why that link is too small

Operators connect several cells to the O-DU pool through **one shared aggregation link** (here a 25 Gbit/s Ethernet link, "25GE"). They deliberately buy a link that is **smaller than the sum of all cells' peaks**, because cells rarely all burst at the same moment. This is called **statistical multiplexing**, and it saves money.

In our setup the 8 cells together could peak at **2.44 times** the link capacity. Most of the time that is fine. When bursts coincide, somebody has to lose data, and the question is **who**.

1.6 Two ways to manage fronthaul load (this is why there is a Part A and a Part B)

- **Part A: make the data smaller.** Compress the IQ data (BFP, low-rank decompositions, and so on). Done earlier in the project.
- **Part B: share the link well.** Given a link smaller than the total peak, decide how much each cell may send. **This is what you are presenting.**

The two meet in one place: the BFP mantissa width sets how many bits one PRB-layer costs, which sets how loaded the link is (Section 8.11).

---

2. The Part B problem in plain words

> Several cells share a fronthaul link that is smaller than their total peak. Every 10 ms a controller must decide how many bits per slot each cell may send, **before** it knows the traffic of the next 10 ms. A cell that gets too little drops bits that wait longer than their deadline; a cell that gets too much wastes capacity another cell needed.

2.1 Why decide in advance, and not every slot?

This is the first question a sceptical examiner asks, so understand it well. If you could decide every 0.5 ms with perfect, fresh information, there would be nothing to predict. In practice you cannot, for three operational reasons:

1. The O-RAN control loop that coordinates several cells (the **near-RT RIC**, talking over the **E2** interface) runs at 10 ms or slower.
2. Traffic shaping in Ethernet switches is not reconfigured every slot.
3. Several O-DU schedulers cannot consult each other inside the sub-millisecond scheduling pipeline.

So a budget is a **reservation** that is held for the whole interval ($T = 10$ ms), and the information used to set it is already $\boldsymbol{\tau = 1}$ **ms old**. The controller must commit for a lead time of $T + \tau$ slots without seeing the future. That is where forecasting comes in.

2.2 The research question

> If each cell's need for the next interval is available as a forecast **distribution** (a range, not a single number), how should the shared link be split between cells that have different deadlines and priorities?

2.3 The answer in three sentences (memorise this)

1. Forecast, for each cell, the **deadline-feasible rate r***: the smallest constant rate at which none of its bits miss their deadline.
2. Forecast it as a **range** (six quantiles from a gradient-boosted tree model), because r^* is skewed: usually small, sometimes very large.
3. Split the link with a closed-form **KKT rule** that cuts every cell at the **same priority-weighted risk** of needing one more bit, so safety margins are **derived** from the forecast, not tuned by hand.

---

3. The system model with real numbers

3.1 Cells

	Low-latency (LL) cells	eMBB cells
How many	3 (LL-0, LL-1, LL-2)	5 (eMBB-0 … eMBB-4)
Deadline D	4 slots = **2 ms**	20 slots = **10 ms**
Priority weight w	**3**	**1**
Traffic character	Burstier, smaller volume	Larger volume

The weight 3 means the operator values a low-latency bit three times an eMBB bit. It is an operator choice, and the results depend on it (Section 8.7).

3.2 The link

- 25 Gbit/s × 0.5 ms = 12.5 Mbit per slot.
- 3 % is reserved for control, management and synchronisation traffic.
- Usable capacity **C = 12.125 Mbit per slot** (24.25 Gbit/s).

3.3 Cost of one PRB-layer (where Part A enters)

$$\beta(b) = (12 \cdot 2 \cdot b + 8) \cdot 14 \cdot \kappa$$

Read it as: 12 subcarriers × 2 numbers (I and Q) × b mantissa bits, plus an 8-bit shared exponent, times 14 symbols per slot, times κ = 1.08 for eCPRI/Ethernet overhead.

- With BFP-9: (216 + 8) × 14 × 1.08 = **β = 3,387 bits**.
- BFP-6 costs 0.68×, BFP-12 1.32×, BFP-14 1.54× of that.

3.4 Peaks and oversubscription

- Cell peak = 273 PRBs × 4 layers × 3,387 bits = 3.70 Mbit per slot ≈ 7.4 Gbit/s.
- 8 cells together = 59 Gbit/s ≈ **2.44 × C**.

3.5 Fronthaul demand of a cell

$$d_i(t) = \lceil a_i(t)/\eta_i(t) \rceil \cdot \beta$$

where $a_i(t)$ is user bits arriving in slot t and $\eta_i(t)$ is user bits per PRB-layer at that moment.

3.6 What is decided

Every T = 20 slots (10 ms) the controller sets a budget r_i (bits per slot) for each cell, such that

$$\sum_i r_i \le C, \qquad 0 \le r_i \le \text{cap}_i = 273 \cdot L_i \cdot \beta$$

Inside the interval, cell i's scheduler may send at most ⌊r_i / β⌋ PRB-layers per slot.

3.7 Queues and violations

- Each cell has a first-in-first-out (FIFO) queue in the O-DU that tracks the age of each bit.
- A bit that has waited longer than its deadline is **dropped and counted as a deadline violation**. A late 7-2x packet is useless because the radio unit's receive window has passed.
- **The link itself never queues.** The budgets always add up to at most C, so congestion shows up as O-DU queueing and drops instead. A unit test checks this every slot.

3.8 What the controller sees

- State that is **τ = 2 slots (1 ms) old**: queue length, age of the oldest bit (head-of-line age), demand history, current spectral efficiency.
- **Never the future.** Also unit-tested.

3.9 The score (chosen before any tuning)

Priority-weighted deadline-violation ratio, written V_w:

$$V_w = \frac{\sum_i w_i \cdot \text{violated bits}_i}{\sum_i w_i \cdot \text{arrived bits}_i} = \frac{3 \times \text{LL dropped} + \text{eMBB dropped}}{3 \times \text{LL arrived} + \text{eMBB arrived}}$$

Lower is better. We also report: violation ratio per class (LL and eMBB), the unweighted ratio, link utilisation, unused allocation, O-DU queueing delay, Jain's fairness index and decision time.

Why weighted? The trade-off between low-latency and eMBB cells is the whole point. An unweighted ratio lets a controller "win" by starving the small LL cells, since they carry fewer bits.

---

4. Key idea 1: the deadline-feasible rate r*

4.1 Intuition

Ask: "How fast must I serve this cell, at a constant rate, so that **no bit waits longer than its deadline**?" That rate is r*.

- If the deadline is **long**, you can smooth a burst out over time, so r* is close to the **average rate**.
- If the deadline is **short**, a burst must be served almost immediately, so r* is close to the **peak burst rate**.

4.2 A tiny worked example

Arrivals over 8 slots: `[0, 0, 8, 0, 0, 0, 0, 0]` (one burst of 8 units). The average rate is 1 unit per slot.

- Deadline D = 1 slot: the 8 units must be gone within 2 slots (the arrival slot plus 1), so r* = 8 / 2 = **4**, which is 4 times the average.
- Deadline D = 7 slots: they have 8 slots, so r* = 8 / 8 = **1**, the average.

That is exactly why the 2 ms LL cells are hard: their r* is their peak 2 ms burst rate, often **2 to 3 times the mean** and strongly right-skewed.

4.3 The formula

For arrivals $a_1, \ldots, a_H$ over the horizon and deadline D:

$$r^*(a, D) = \max_{1 \le s \le e \le H} \frac{\sum_{t=s}^{e} a_t}{e - s + 1 + D}$$

In words: look at **every window** of slots [s, e]. The bits arriving in that window must be served by slot e + D, which gives (window length + D) slots of service. The worst window sets the rate. This is the network-calculus "service curve" argument (Le Boudec and Thiran, 2001). In the simulator r* is tight to within one PRB-layer, and a test checks this.

4.4 What every deadline-aware controller asks for

$$\text{need}_i = B_i + r_i^*, \qquad B_i = \frac{\text{backlog}_i}{\text{slack}_i}$$

B_i is the rate needed to clear what is **already** in the queue before it expires. r_i^* covers what has **not arrived yet**. All deadline-aware controllers in the study use this same structure; they differ **only** in how they estimate r_i^* and how they split a shortage. That is what makes the comparison fair.

4.5 Why one number is a bad basis for a reservation

r* is skewed: usually low, sometimes very high. If you reserve the **median**, you under-serve every burst. If you add a **fixed safety margin** to everyone, you waste capacity on cells that did

not need it. On validation data, every positive margin (+25 %, +50 %) made every baseline **worse**. Hence: forecast a **range**.

---

5. Key idea 2: forecast a range, not a number

5.1 What a quantile is

The **α-quantile** of an uncertain quantity is the value it stays below with probability α. The 50 % quantile is the median. The 95 % quantile is a "nearly worst case". Six quantiles (5, 25, 50, 75, 90, 95 %) give a picture of the whole range. Join them with straight lines and you have an approximate cumulative distribution function F (and its inverse F^{-1}).

5.2 The model: gradient-boosted trees

- **Gradient boosting** builds many small decision trees one after another, each fixing the errors of the previous ones.
- With a **quantile loss** (the "pinball loss"), a model learns to predict a chosen quantile instead of the average.
- We train **one model per quantile level** (6 models), each with **300 trees of 15 leaves**, using scikit-learn's `HistGradientBoostingRegressor`.

5.3 What it predicts and from what

- **Target:** r* of the cell for the coming horizon, divided by the cell's peak (so all cells are on the same 0 to 1 scale).
- **28 features, all from the past only:** the last 6 horizon means (and their mean, std and max), two exponentially weighted moving averages, the last 4 slot demands, r* of the last 3 horizons, relative spectral efficiency, D/H, and a one-hot code of the cell. Normalised by configuration constants, so nothing is fitted on test data.
- **Training data:** 16 traces × 16,000 slots ≈ 101k rows. **12 seconds** on a CPU.

5.4 Seeds keep training, tuning and testing separate

Use	Random seeds
Training the forecaster	1000–1015
Validation (tuning all baselines' knobs)	2000–2003
Test (the reported results)	3000–3019

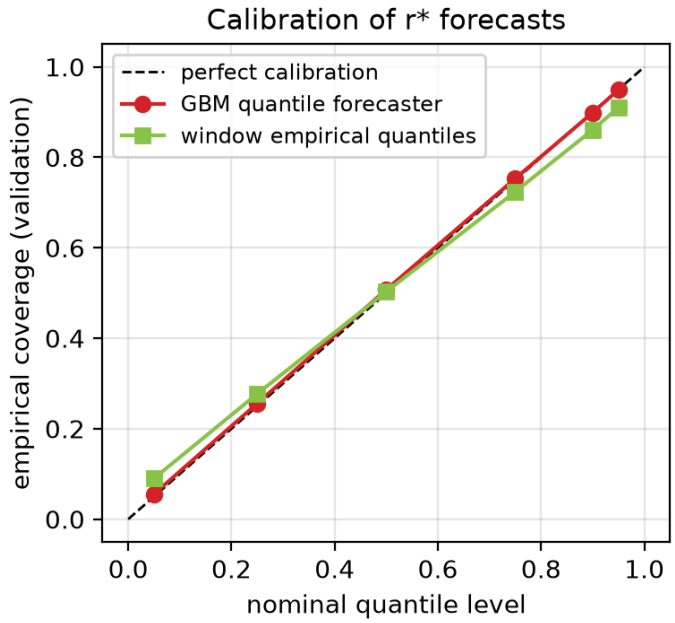
They never overlap. The forecaster is frozen before any test run.

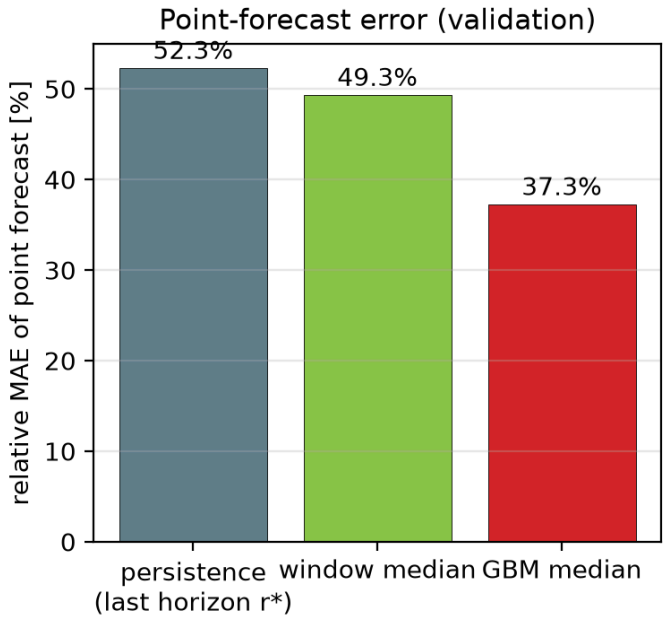
5.5 Is the forecast any good?

On validation data:

Nominal quantile	5 %	25 %	50 %	75 %	90 %	95 %
How often the true value fell below it	5.6 %	25.6 %	50.7 %	75.3 %	89.9 %	94.9 %

That is **well calibrated** (each within about one point of nominal). Median relative error: **37 %** for the boosted trees, versus 49 % for a "median of the last 30 intervals" and 52 % for "assume the last value" (persistence). Pinball loss 0.0224 versus 0.0293.


Calibration of r* forecasts
perfect calibration
GBM quantile forecaster
window empirical quantiles
empirical coverage (validation)
nominal quantile level
0.0
0.2
0.4
0.6
0.8
1.0


Point-forecast error (validation)
relative MAE of point forecast [%]
52.3%
49.3%
37.3%
persistence
(last horizon r*)
window median
GBM median
0
10
20
30
40
50

6. Key idea 3: the KKT (newsvendor) allocation rule

6.1 The newsvendor problem in one paragraph

A newspaper seller must decide in the morning how many papers to stock, without knowing how many will sell. Stock too few and you lose sales; stock too many and they go to waste. The classic answer: stock up to the quantile of demand where the chance of selling one more paper exactly balances its cost. With **several products and one shared budget** (the "constrained multi-item newsvendor", Hadley and Whitin, 1963), all products are stocked up to the point where the **value of one more unit is the same** for all of them.

Our version: products = cells, stock = budget, shared budget = link capacity.

6.2 The objective

For one interval, with forecast distribution F_i of r_i^*:

$$\max_{r} \sum_i w_i \, \mathbb{E}\big[\min(B_i + r_i^*,\ r_i)\big] \quad \text{s.t.} \quad \sum_i r_i \le C,\ 0 \le r_i \le \text{cap}_i$$

"Maximise the priority-weighted amount of need that is actually covered, on average under the forecast." The expected served amount $\mathbb{E}[\min(\cdot, r)]$ is **concave** in r (each extra bit of budget is worth less than the previous one). Its slope is $w_i(1 - F_i(r - B_i))$: the weight times the **probability that the next bit of budget is actually needed**.

6.3 The solution

Because the problem is separable and concave, the **KKT conditions** (the standard optimality conditions for constrained problems; KKT = Karush–Kuhn–Tucker) give the global optimum in closed form:

$$\boxed{r_i(\lambda) = \text{clip}\Big(B_i + F_i^{-1}\big(1 - \lambda / w_i\big),\ 0,\ \text{cap}_i\Big)}$$

- **λ** is a single number (a "price" of link capacity), the same for every cell. It is found by **60 steps of bisection** so that the budgets add up exactly to C.
- If the link is not full (the constraint is not binding), λ = 0 and every cell gets its full predicted range, with the surplus shared out as headroom.
- F^{-1} is the straight-line interpolation through the six forecast quantiles.
- Cost: O(N·K + N·iterations), a few tens of microseconds for 8 cells.

6.4 What the formula means (the sentence to say on stage)

> **Every cell is cut at the same priority-weighted risk.** A cell with a wide (uncertain) forecast or a high priority automatically gets more margin above its median. A cell with a confident forecast gets about its median. The backlog is always funded first. **Margins are derived, not tuned.**

6.5 A toy example

Two cells, link C = 10.

- LL cell (w = 3): its level is 1 − λ/3.
- eMBB cell (w = 1): its level is 1 − λ.

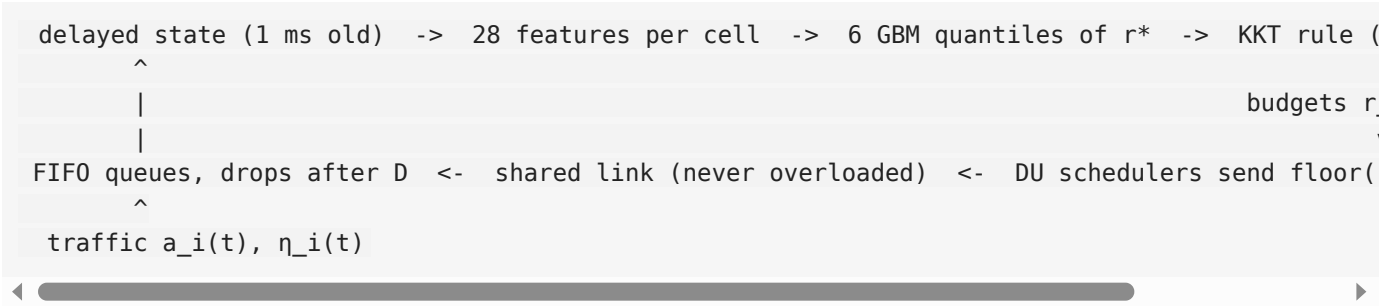
Say bisection finds λ = 0.6. Then the LL cell is funded up to its **80 %** forecast quantile (1 − 0.6/3 = 0.8) and the eMBB cell up to its **40 %** quantile (1 − 0.6 = 0.4). If those quantiles are 4 and 6, the budgets are 4 + 6 = 10 = C. The high-priority, bursty cell gets a generous margin; the tolerant cell runs a bit below its median. Note that if λ ≥ w for a cell (here λ ≥ 1 for eMBB), its level drops to 0 or below and it gets only its backlog. This "saturation" explains a finding of the extended study (Section 9.5).

6.6 The four propositions (proved in `docs/fhlm/method.md`, checked by tests)

#	Name	In plain words
1	Feasibility	Whatever the forecast, the budgets fit within the caps and add up to at most C. The link is never overloaded.
2	Equal weighted tail probability	At the optimum every cell has the same w_i × P(needs more). With a single-number forecast, the rule reduces to filling cells in priority order, which is why the point-forecast controller is the fair comparison.
3	Backlog first	Bits already waiting are funded before any uncertain future demand of an equal- or lower-weight cell.
4	Robustness to a wrong forecast	If the forecast distribution is off by at most ε everywhere (Kolmogorov distance), the loss is at most 2·ε·w_max·C. It grows **linearly** in ε, so a somewhat wrong forecast degrades gently.

Be precise about what these are: properties of the **one-interval** problem. There is **no** claim about closed-loop optimality or queue stability.

6.7 The whole control loop



Edge cases: if the forecast is not a finite number, fall back to window quantiles; zero demand gets zero budget plus a share of headroom; a link that is not full gives every cell its full predicted range.

Calibrated variant (an ablation): an online rule called **adaptive conformal inference (ACI)** (Gibbs and Candès, 2021) shifts each cell's quantile level up after a miss and down after a hit. It is reported as an ablation because it **did not help** (Section 8.5).

---

7. How the experiments were run

7.1 The controllers compared (all knobs tuned on validation seeds)

Controller	How it estimates r*	How it splits the link	Why it is in the study
Static equal share	does not	C / 8 each	Floor: no intelligence
Reactive proportional	running average of demand (EWMA), ignores deadlines	proportional	Simple, common practice
MaxWeight	uses priority × queue length, head-of-line-age urgency	largest weight first	Textbook queue-based scheduler
Deadline-aware reactive	last observed r* (persistence)	deadline-weighted water-fill	**Strong non-ML baseline**: knows deadlines
Point forecast	GBM **median**	deadline-weighted water-fill	**Key ablation**: same model, one number
Fixed global quantile	GBM quantile, one tuned level (0.75) for everyone	water-fill	Tests whether "just use a quantile" is the trick
Proposed (window)	empirical quantiles of the last 30 intervals, **no ML**	KKT rule	Tests whether ML is needed
Proposed	**GBM quantiles**	**KKT rule**	The method
Proposed + ACI	GBM quantiles + online calibration	KKT rule	Calibration ablation
Oracle water-fill	**true** r* (impossible in practice)	water-fill	Reference: perfect single number
Oracle + KKT	**true** r*	KKT rule	Reference: how much is forecast error

"**Water-fill**" means: give every cell what it asks for if possible and redistribute what is left; under shortage, split in proportion to weight × demand.

Fair tuning: margins {0, 0.25, 0.5}, running-average windows {20, 60} and persistence windows {1, 3, 6} were tuned on validation seeds. **Margin 0 was best for every baseline**: inflating a single estimate hurts when a shortage is split proportionally, because over-claiming cells get more. Every baseline redistributes unused capacity. The deadline-aware reactive controller uses the same weights, backlog rate and water-fill as the point-forecast one.

7.2 Traffic: two independent families

Family S (synthetic, deliberately bursty). Per cell: a smooth Gamma background, plus ON/OFF bursts with **heavy-tailed (Pareto) durations** ($\alpha_{on} = 1.5$, $\alpha_{off} = 1.8$, following Willinger et al. 1997, the classic model of bursty internet traffic), plus slow load changes about every second, plus drifting spectral efficiency. Five scenarios:

Scenario	Mean load	What is special
low	0.50	
moderate	0.65	
high	0.80	the main stress test
flash crowds	0.80	short 1.8× overloads 15 % of the time
shift (held-out)	0.80	bursts 1.5× stronger and longer, heavier tail; **never seen in training or tuning**

Family G (standards-based: nothing hand-designed). Built only from 3GPP models: the TR 38.901 urban-macro channel (7 interfering sites, shadowing, Jakes fading at 3 km/h), TS 38.214 link adaptation (CQI table), FTP Model 3 file downloads for eMBB cells (TR 36.814), XR video at 60 fps for LL cells (TR 38.838). Fronthaul demand is the output of a round-robin PRB scheduler. Own forecaster, own tuning. Scenarios: moderate 0.65 and high 0.80.

Why two families? A reviewer would say "you designed the traffic your method likes". Family G answers that.

7.3 Protocol and statistics, explained simply

- **20 test seeds × 20,000 slots (10 s each)** after 1,000 warm-up slots.
- **Every controller sees exactly the same traffic and the same delayed observations, seed by seed.** This lets us compare them **in pairs**, which removes the noise from "this seed happened to be burstier".
- **Paired bootstrap 95 % confidence interval.** Resample the 20 per-seed differences many times to see how much the average reduction could vary.
- **Wilcoxon signed-rank test.** Checks whether the proposed rule is consistently better across seeds, without assuming a bell curve.
- **Holm correction.** Because we compare against many baselines, we tighten the p-values so we do not "find" a win by luck.
- Main comparison = 10 controllers × 5 scenarios × 20 seeds = **1,000 runs** (plus 100 oracle+KKT runs and 480 Family G runs).

---

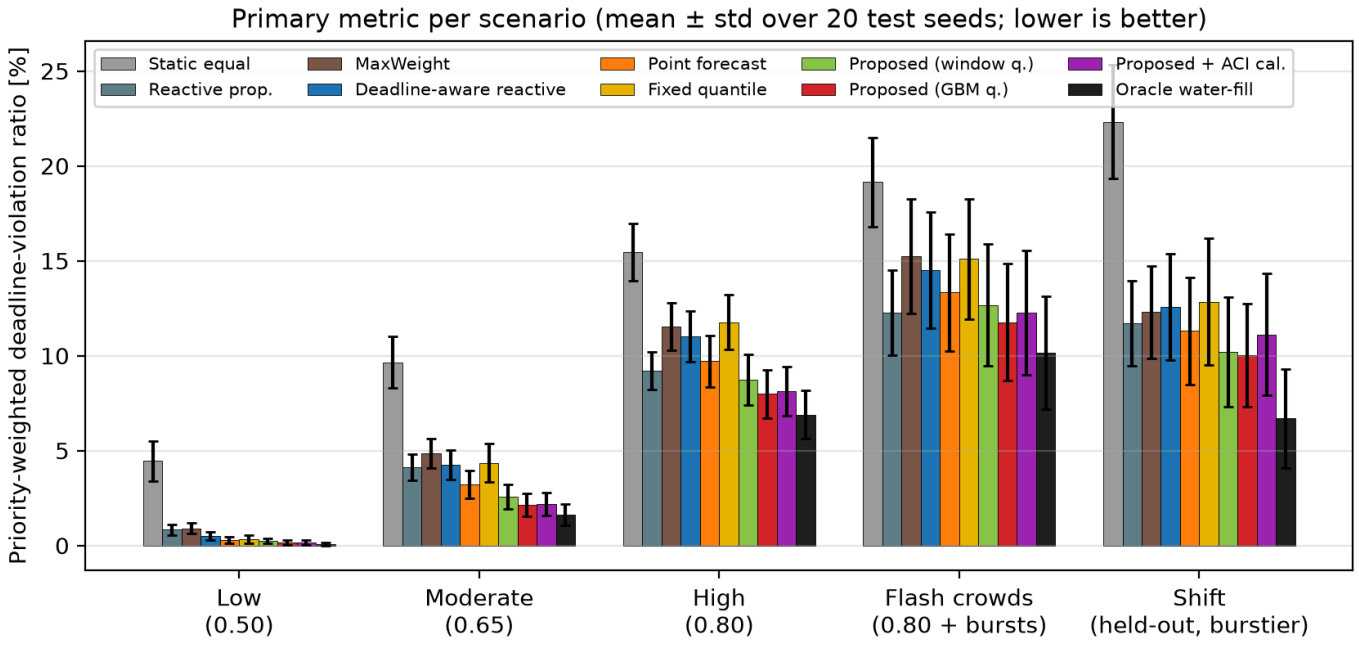
8. Results

8.1 Main result, Family S (V_w in %, mean ± std over 20 seeds; lower is better)

Method	low 0.50	moderate 0.65	high 0.80	flash crowds	shift (held-out)
Static equal share	4.45 ± 1.05	9.65 ± 1.36	15.46 ± 1.51	19.15 ± 2.34	22.31 ± 2.99
Reactive proportional	0.83 ± 0.27	4.11 ± 0.69	9.20 ± 0.99	12.27 ± 2.23	11.70 ± 2.24
MaxWeight	0.90 ± 0.28	4.85 ± 0.79	11.54 ± 1.27	15.24 ± 3.02	12.31 ± 2.44
Deadline-aware reactive	0.52 ± 0.21	4.24 ± 0.78	11.03 ± 1.33	14.51 ± 3.06	12.56 ± 2.81
Point forecast (GBM median)	0.27 ± 0.17	3.23 ± 0.73	9.70 ± 1.34	13.33 ± 3.09	11.30 ± 2.81
Fixed global quantile (0.75)	0.34 ± 0.23	4.35 ± 1.02	11.76 ± 1.44	15.09 ± 3.17	12.85 ± 3.32
Proposed, window quantiles (no ML)	0.25 ± 0.13	2.57 ± 0.64	8.72 ± 1.34	12.67 ± 3.19	10.22 ± 2.88
Proposed (GBM quantiles + KKT)	**0.15 ± 0.13**	**2.15 ± 0.60**	**8.00 ± 1.27**	**11.76 ± 3.08**	**10.04 ± 2.71**
Proposed + ACI calibration	0.17 ± 0.13	2.17 ± 0.61	8.14 ± 1.30	12.27 ± 3.26	11.12 ± 3.22
Oracle water-fill (not implementable)	*0.07*	*1.62*	*6.89*	*10.16*	*6.69*

Headline: the proposed rule is the best implementable controller in all 5 scenarios.

Why are the standard deviations large and overlapping? Because they mostly reflect burst timing, which every controller shares on the same seed. The paired comparison below is the right way to read the result.

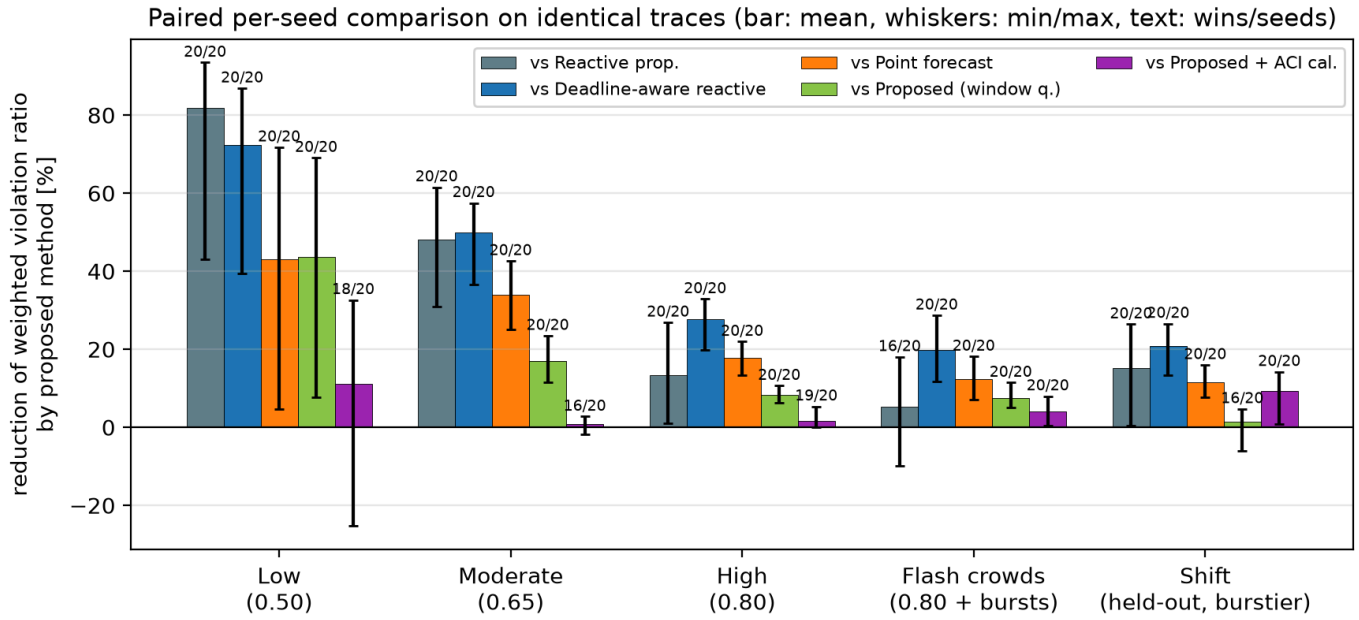


8.2 Paired, seed-by-seed reduction by the proposed rule (Family S)

Mean relative change of V_w, seeds won out of 20. ** = Holm-corrected Wilcoxon p < 0.01, * = p < 0.05.

Proposed vs.	low	moderate	high	flash crowds	shift
Reactive proportional	−81 % 20/20**	−48 % 20/20**	−13 % 20/20**	**−4 % 16/20***	−14 % 20/20**
MaxWeight	−83 % 20/20**	−56 % 20/20**	−31 % 20/20**	−23 % 20/20**	−18 % 20/20**
Deadline-aware reactive	−70 % 20/20**	−49 % 20/20**	−27 % 20/20**	−19 % 20/20**	−20 % 20/20**
Point forecast	−43 % 20/20**	−33 % 20/20**	−18 % 20/20**	−12 % 20/20**	−11 % 20/20**
Fixed global quantile	−54 % 20/20**	−50 % 20/20**	−32 % 20/20**	−22 % 20/20**	−22 % 20/20**
Window quantiles (no ML)	−37 % 20/20**	−16 % 20/20**	−8 % 20/20**	−7 % 20/20**	**−2 % 16/20****
+ ACI calibration	−9 % 18/20**	−1 % 16/20**	−2 % 19/20**	−4 % 20/20**	−10 % 20/20**

- **Strongly supported:** against the point forecast, the fixed quantile, MaxWeight and the deadline-aware reactive controller, the proposed rule wins in every scenario (including the held-out shift), **on every seed**, p < 0.01.
- **Weak spot:** against plain reactive proportional under flash crowds, only −4 % (16/20 seeds, p = 0.03). There, proportional starves the LL cells (17.8 % LL violations vs 4.6 %) but serves eMBB much better (7.1 % vs 18.3 %), so the weighted score barely moves.

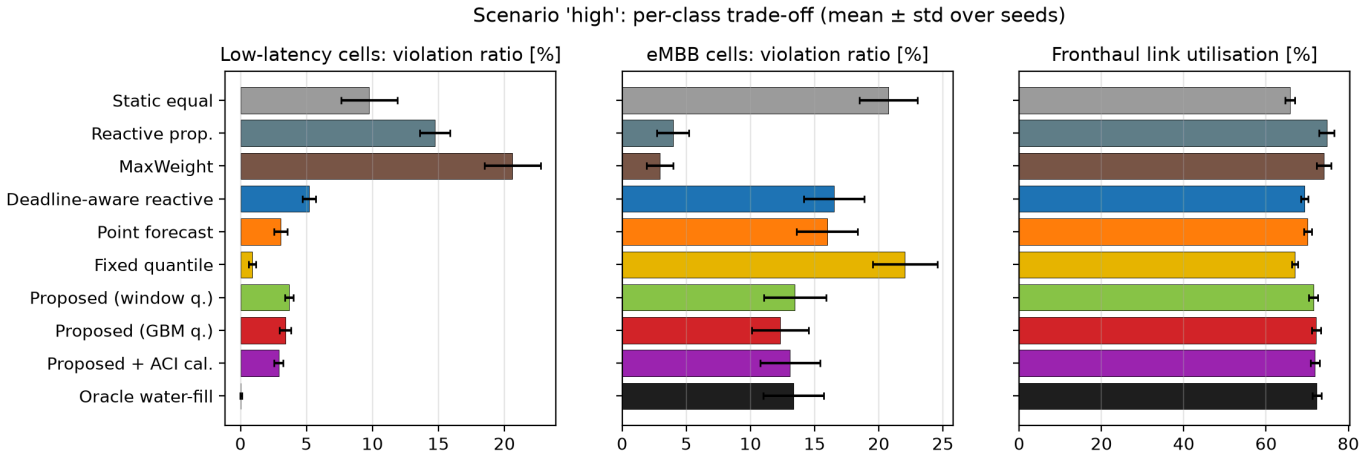


8.3 Where the gain comes from (scenario high, 20 seeds)

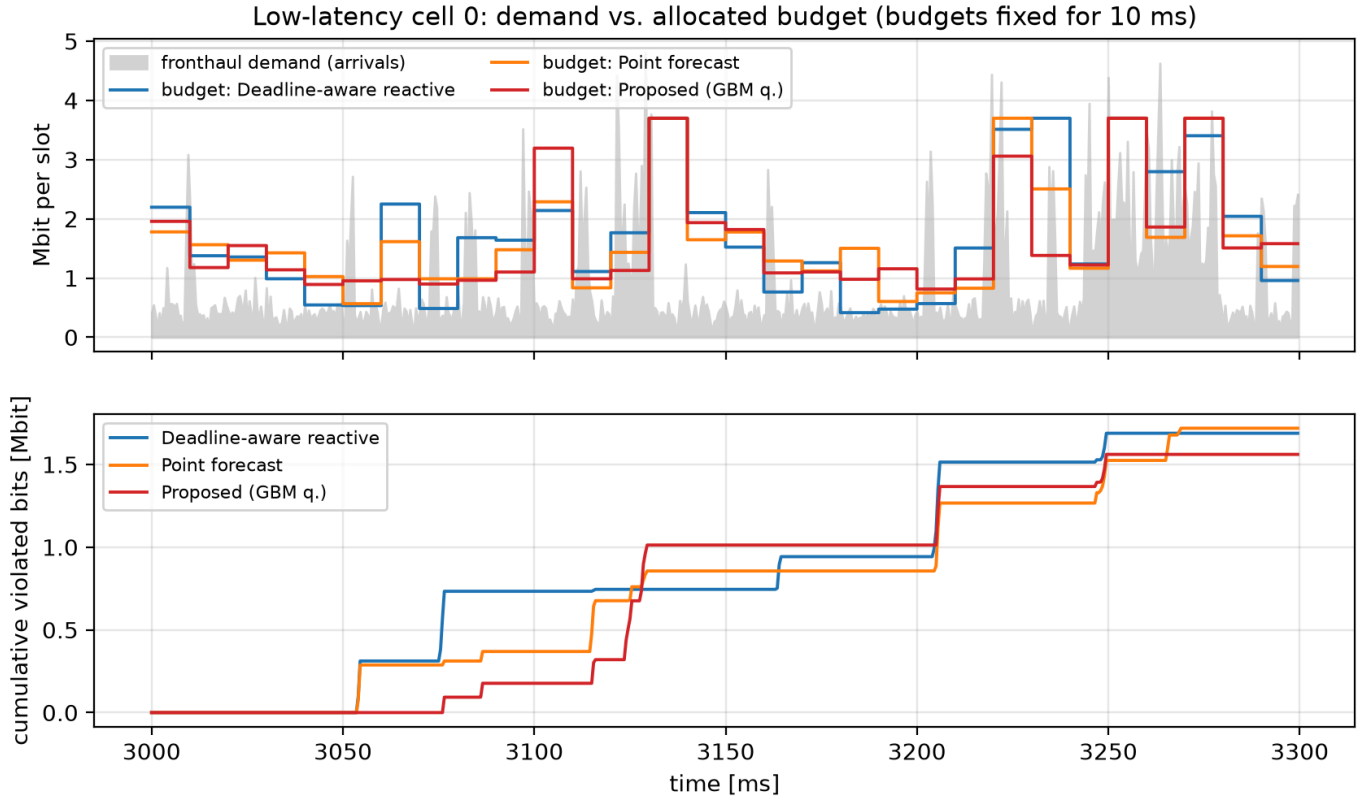
Method	LL violations	eMBB violations	Utilisation	Jain fairness
Reactive proportional	14.7 %	4.0 %	74.7 %	0.996
MaxWeight	20.6 %	2.9 %	74.1 %	0.991
Deadline-aware reactive	5.2 %	16.5 %	69.3 %	0.995
Point forecast	3.1 %	16.0 %	70.1 %	0.994
Fixed global quantile	0.9 %	22.0 %	67.0 %	0.985
Proposed	3.4 %	**12.3 %**	**72.2 %**	0.997
Oracle water-fill	0.04 %	13.4 %	72.3 %	0.994

The story in this table:

1. **Deadline-unaware controllers sacrifice the LL cells** (14.7 %, 20.6 %). A large queue is not the same as an urgent queue.
2. **Every deadline-aware controller protects LL** and pays with eMBB.
3. **Among them, the proposed rule loses the least eMBB and has the highest utilisation**, because it adds margin only where the forecast is wide (LL cells at burst onset) and lets confident cells sit at their median.
4. **One global level (fixed quantile) over-protects LL** (0.9 %) and wastes 33 % of the allocation, giving the worst eMBB loss. So the per-cell level chosen by the link constraint is what matters, not merely "using a quantile".
5. Even the oracle loses 13 % of eMBB bits: budgets fixed for 10 ms cannot follow heavy bursts. That is why absolute numbers are high.
6. Fairness is > 0.98 for everyone, so it is not a differentiator.



8.4 The rule in action: one LL cell



How to read it: grey is the demand of one LL cell; the step lines are budgets held for 10 ms. The reactive controller (blue) is **one interval late** at every burst: it raises the budget after the burst has already happened. The proposed rule (red) raises the budget where the forecast range is **wide** (burst onset). The bottom panel is cumulative violated bits: the proposed rule ends with the fewest over this trace. (Honest note: it is not ahead at every moment; around 3,130 ms it is briefly worse. It wins over the whole trace.)

8.5 Ablations: which piece carries the gain?

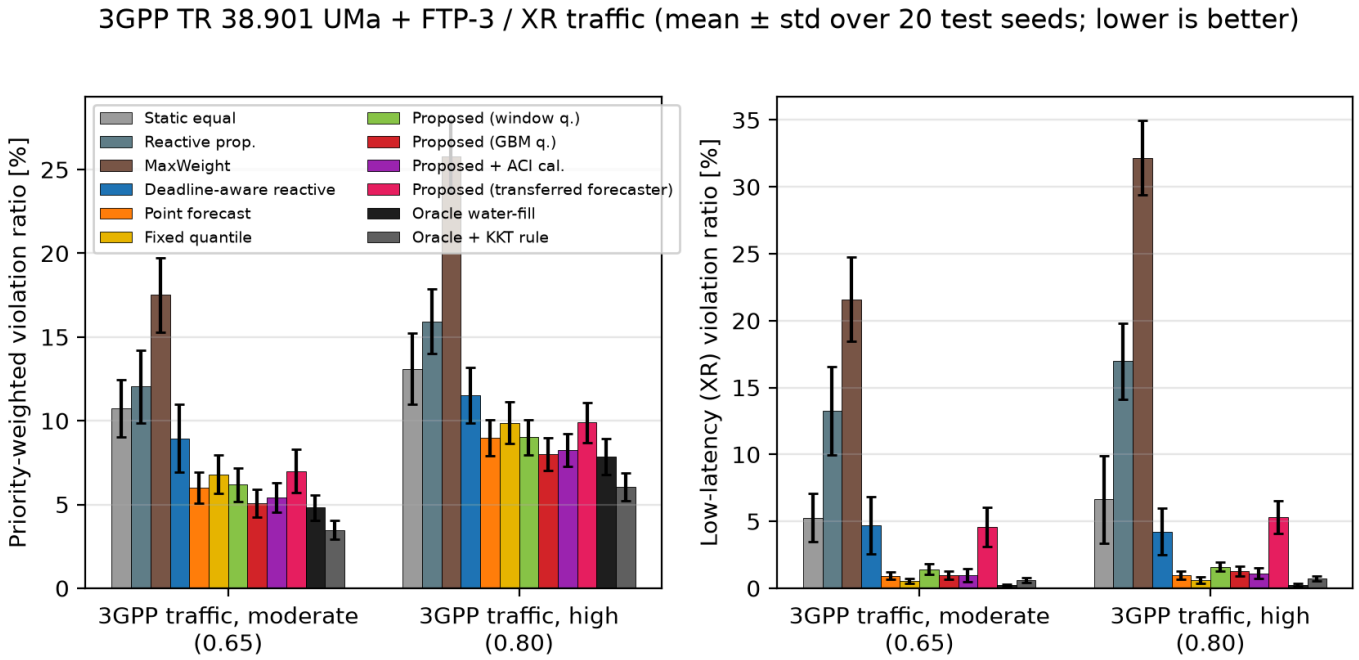
1. **Range vs single number** (same model, same rule family): −11 % to −43 %, 20/20 seeds everywhere. **Representing uncertainty matters.**
2. **Per-cell KKT level vs one tuned global level** (same forecast): −22 % to −54 %. **The level must be chosen per cell by the link constraint.**

3. **ML vs non-ML quantiles** under the same KKT rule: the boosted trees add −7 % to −37 % on normal traffic, but only −2 % under the held-out shift. **Most of the gain is the rule (the r* target plus the KKT split), not the model. ML is useful, not necessary.** The window-quantile version is the robust fallback.
4. **Online ACI calibration: no gain** (1–9 % worse on normal traffic, 10 % worse under shift). Why: in a **coupled** allocation, LL misses are partly **caused by the shortage split**, not by a biased forecast. Calibration then inflates LL reservations and starves eMBB (under shift: LL 4.7 % vs 6.7 %, but eMBB 17.2 % vs 13.2 %). Per-stream conformal correction does not carry over to coupled cells. This is a negative result, reported honestly.

8.6 Family G (3GPP traffic): does it reproduce?

Method	moderate 0.65	high 0.80	unweighted (high)
Static equal	10.75	13.10	19.2
Reactive proportional	12.05	15.94	15.0
MaxWeight	17.51	25.79	19.7
Deadline-aware reactive	8.95	11.53	18.5
Point forecast	6.00	8.98	16.6
Fixed quantile	6.80	9.88	18.7
Window quantiles	6.18	9.01	16.0
Proposed	**5.08**	**8.01**	**14.4**
Transfer (forecaster trained on Family S)	6.99	9.89	14.3
Oracle + KKT	*3.47*	*6.06*	*11.1*

- **Yes, it reproduces** on traffic we did not design: best implementable at both loads, 20/20 seeds.
- At high load: −11 % vs point forecast, −31 % vs deadline-aware reactive, −50 % vs reactive proportional.
- Here it is also best under the **unweighted** ratio.
- The forecaster trained on Family S and applied unchanged to 3GPP traffic still beats every reactive controller, but not the in-family point forecast.
- Deadline-unaware controllers do even worse than static sharing here: an FTP file is a burst at full cell rate and XR frames are periodic, so "large queue" and "urgent queue" are very different.



8.7 Honest caveat: the gain is a priority trade

Unweighted violation ratio, Family S (%):

Method	moderate	high	flash crowds
Reactive proportional	**2.44**	**6.55**	**9.64**
MaxWeight	2.73	7.18	10.44
Point forecast	3.93	12.88	17.62
Proposed (run with w_LL = 3)	2.50	10.19	15.08

If every bit counts the same, plain proportional sharing loses **fewer total bits** at moderate and high load in Family S, because it sacrifices the small LL cells to serve eMBB volume.

LL-weight sweep (S high, every controller told the new weight, V_w %):

w_LL	1	2	3 (default)	5
Reactive proportional	**6.55**	**8.13**	9.20	10.56
Point forecast	6.77	8.90	9.70	10.57
Proposed	6.58	8.35	**8.00**	**6.93**

The rule ties with proportional at w_LL = 1 and pulls ahead from w_LL = 3. In Family G at w_LL = 1 it is clearly **worse** than the point forecast (15.9 % vs 10.4 %): 3GPP demand is often exactly zero in a horizon (no file, no frame), so at a low common risk level the newsvendor reserves nothing beyond backlog for many cells, an all-or-nothing allocation. With a perfect forecast (oracle + KKT) it would win at every weight, so that loss is forecast error.

Scope of the claim: operators who value a low-latency bit at least 2–3× an eMBB bit. Say this before you are asked.

8.8 How much is still forecast error? (oracle + KKT)

Scenario	Proposed	Oracle + KKT	Gap (share of the proposed rule's loss)
S low	0.15	0.05	70 %
S moderate	2.15	0.88	59 %
S high	8.00	4.77	40 %
S flash crowds	11.76	8.10	31 %
S shift	10.04	5.84	42 %
G moderate	5.08	3.47	32 %
G high	8.01	6.06	24 %

- Oracle + KKT is the same rule fed a perfect forecast of r*. The gap is what a better forecaster could still buy: **24–40 % at high load**. The rest is the unavoidable cost of committing budgets for 10 ms.
- At T = 2 ms, oracle + KKT reaches 1.91 % against 6.89 % for oracle water-fill with **the same perfect information**. **The allocation rule matters as much as the forecast.**
- So "oracle" is a **reference, not an upper bound**. Never call it a bound.

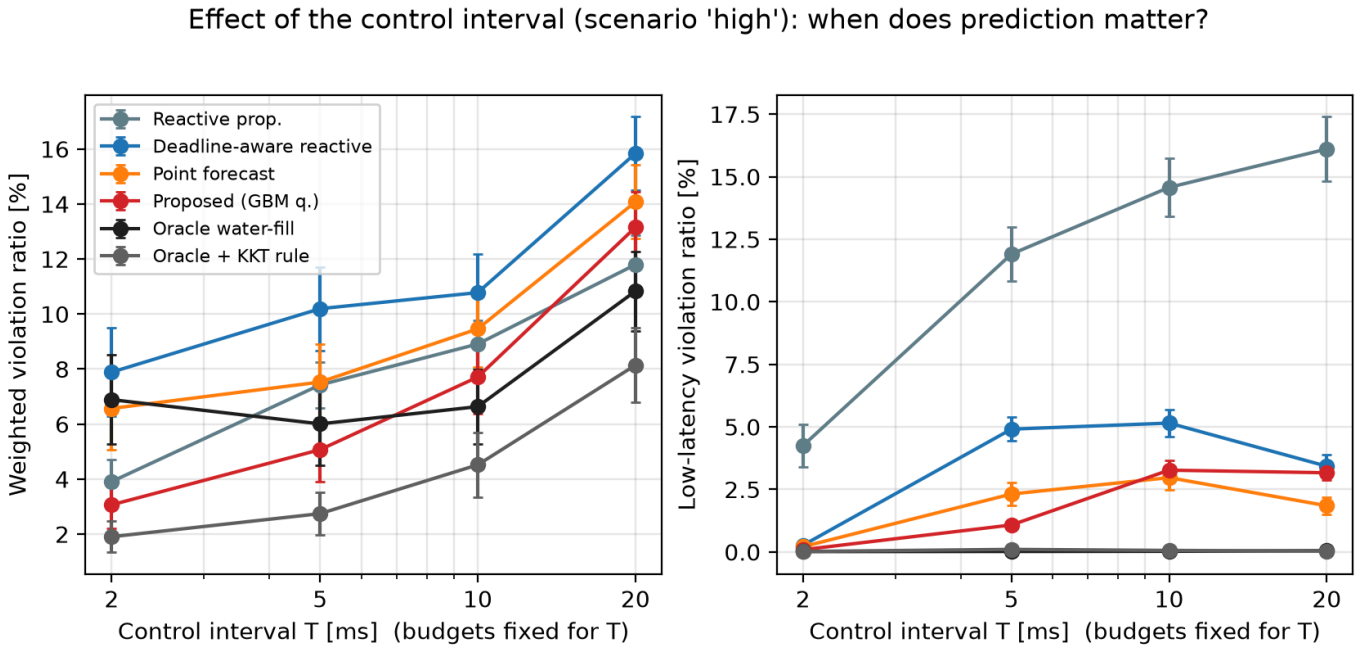
8.9 Sensitivity: control interval T and telemetry delay τ

These sweeps use scenario high with **10 seeds** and forecasters retrained for each setting. That is why the T = 10 ms row (7.71 %) differs slightly from the main table (8.00 %, 20 seeds). If someone spots this, that is the reason.

T	Reactive prop.	Deadline-aware	Point forecast	Proposed	Oracle water-fill
2 ms	3.91	7.89	6.57	**3.07**	6.89
5 ms	7.42	10.20	7.53	**5.07**	6.01
10 ms	8.91	10.78	9.46	**7.71**	6.63
20 ms	**11.80**	15.84	14.08	13.15	10.83

τ	Reactive prop.	Proposed	Proposed vs reactive prop.
0 ms	8.16	6.85	−16 %
1 ms (default)	8.91	7.71	−13 %
2.5 ms	9.86	8.77	−11 %
5 ms	11.30	11.07	−2 %

- **Short interval:** the rule even beats the oracle water-fill (−55 % at 2 ms).
- **T = 20 ms (5× the LL deadline):** reserving the LL peak rate for the whole interval over-reserves, and plain proportional wins (+11 %).
- **τ = 5 ms (2.5× the LL deadline):** the advantage over proportional shrinks to 2 %. The ordering against the deadline-aware controllers stays the same at every τ.
- **Regime of validity:** intervals of up to a few multiples of the shortest deadline, with reasonably fresh telemetry.



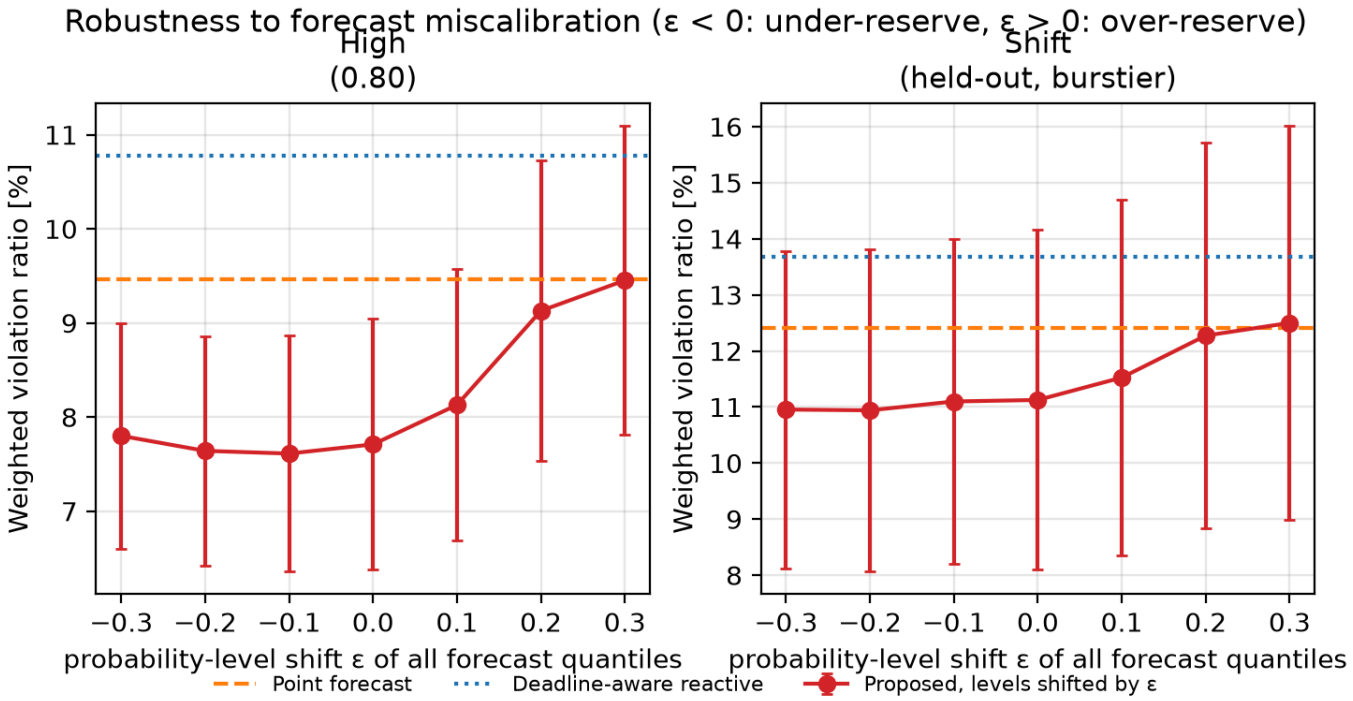
8.10 Robustness to a miscalibrated forecast (Proposition 4 in practice)

Every forecast quantile level is deliberately shifted by ε before allocation (ε < 0: under-reserve; ε > 0: over-reserve):

ε	−0.3	−0.1	0	+0.1	+0.3
high	7.80	7.61	7.71	8.13	9.45
shift	10.96	11.10	11.13	11.53	12.50

Point forecast: 9.46 / 12.42. Deadline-aware reactive: 10.78 / 13.68.

- **Under-reserving is almost harmless**: a common shift of every cell is absorbed by λ.
- **Over-reserving degrades gradually** (+23 % at ε = +0.3, high load).
- Even then, the rule only falls back to the point-forecast level and still beats the deadline-aware reactive controller.



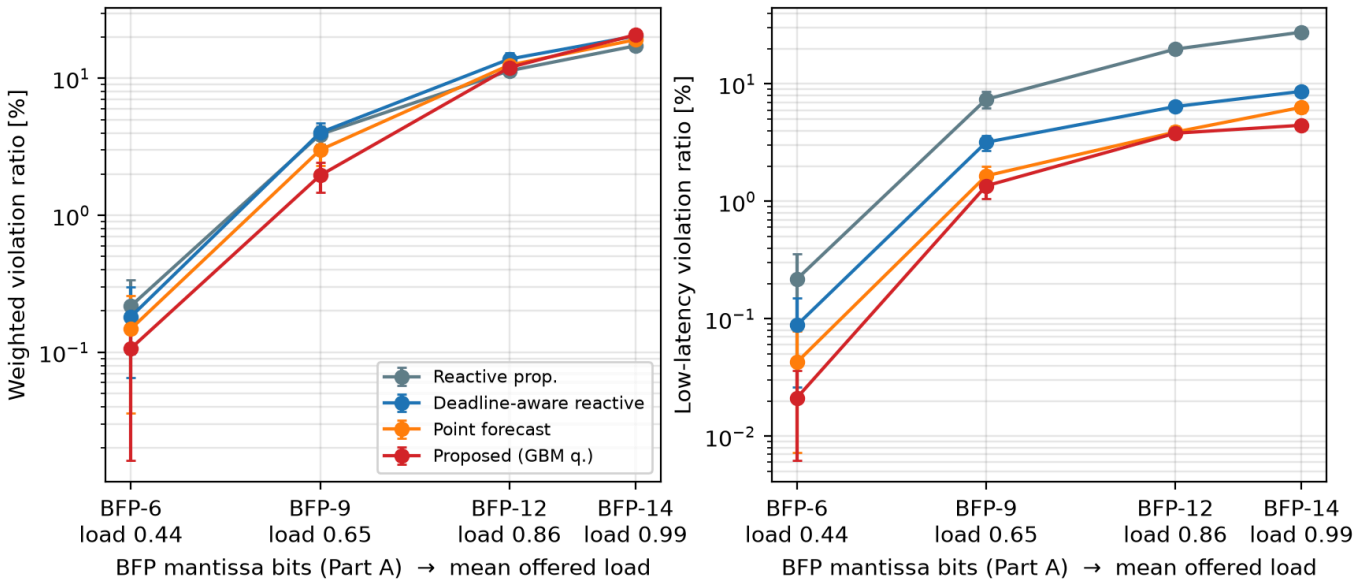
8.11 Part A × Part B: compression width vs allocation rule

Same user traffic (scenario moderate, 10 seeds). Only the BFP mantissa width changes, which changes β and therefore the load:

BFP width	Load on link	Reactive prop.	Point forecast	Proposed
6 bits	0.44	0.22	0.15	**0.11**
9 bits (default)	0.65	3.90	3.00	**1.96**
12 bits	0.86	**11.37**	12.50	12.01
14 bits	0.99	**17.29**	19.16	20.81

- **Compression sets the regime; allocation decides the loss inside it.**
- At 0.44 load almost nothing is lost whatever you do.
- At 0.65 the proposed rule loses **about half** as much as the reactive controllers. **Allocation matters most between about 0.5 and 0.8 load.**
- At 0.86 proportional is already slightly better; at 0.99 no split helps. You need more compression or more capacity.
- Not modelled: the signal-quality cost of fewer bits. Part A's NMSE curves cover that side.

Same user traffic, different compression: compression sets the load, allocation decides the loss



8.12 Decision overhead (Python, 4-core CPU, per 10 ms control interval)

Controller	Time
Static	0.02 ms
Reactive proportional	0.04 ms
MaxWeight	0.05 ms
Deadline-aware reactive	0.09 ms
Window quantiles	2.2 ms
Point forecast	2.5 ms
Proposed	**3.5 ms**

- The KKT rule itself takes **less than 0.1 ms**. The cost is building features and evaluating 6 × 300 trees for 8 cells.
- We wrote a vectorised numpy tree traversal (1.8 ms, versus 12 ms for scikit-learn's `predict`).
- A real O-DU would need a compiled model (C++/numba) or the no-ML window fallback. We do not claim a measured compiled number.

---

9. The extended study (theory and stronger controllers)

This was added after the NCC 2027 conference submission (6 pages), to turn it into an 11-page journal-length manuscript (`paper/extended/`). It asks three questions.

9.1 What is the real control problem?

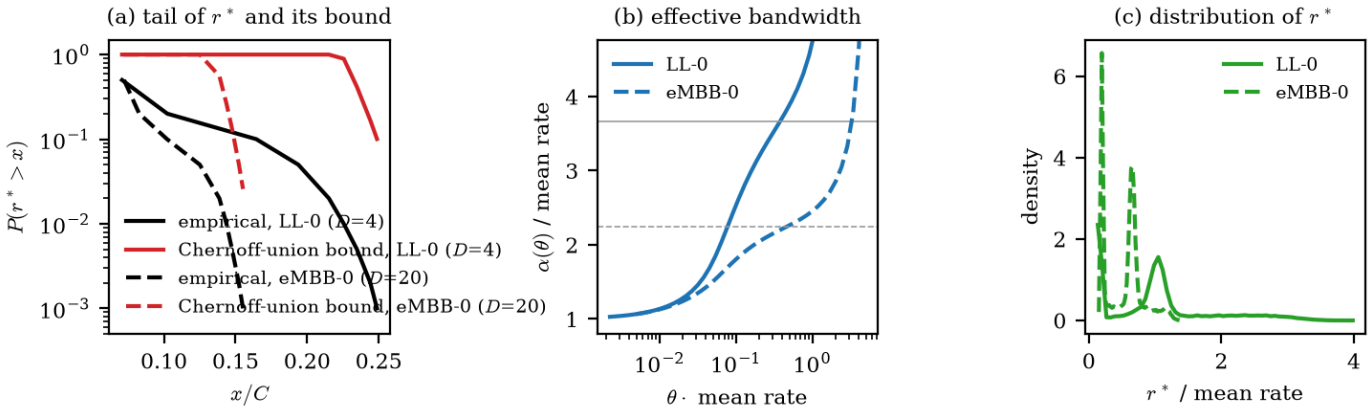
Formally it is a **POMDP** (partially observed Markov decision process): the state is every queue's age profile, the hidden traffic state and the spectral efficiency; you act every T slots on information τ slots old; you want to minimise the total weighted drops. That is **intractable** (hundreds of continuous state dimensions, unknown traffic law, actions held for T slots). There are three standard ways to approximate it, and all three were implemented:

1. **One-step stochastic programs:** the KKT newsvendor rule (ours), on a learned or model-based distribution.
2. **Lyapunov drift / back-pressure:** MaxWeight, and **predictive MaxWeight** (Huang et al. 2016), which adds a look-ahead using the forecast median to the queue length.
3. **Direct policy search:** a 3-parameter policy learned from reward with the **cross-entropy method (CEM)**, a small analogue of a deep reinforcement learning xApp.

A third traffic family, **M**, was added: Markov-modulated Gamma sources with three hidden phases (idle, normal, burst), and a **semi-Markov held-out shift** with Pareto sojourns that breaks every Markov model.

9.2 Theory of r* for Markov traffic (what to say, without the algebra)

- An exact expression for the "window log-moment generating function" of the traffic, and the **effective bandwidth** (a classical quantity that runs from the mean rate to the peak rate).
- A **Chernoff-union tail bound** on P(r* > x).
- **Large-deadline decay:** as the deadline grows, the tail of r* decays like classical buffer-overflow probabilities. This links r* to textbook queueing theory.
- A pathwise **shortfall sandwich**, valid for any traffic: $(D + 1)(r^* - r)^+ \le$ dropped volume $\le \Sigma\,(a_t - r)^+$. So the newsvendor objective is a **lower bound** on the violated volume.
- **Validation:** the sandwich is never violated; the bound is valid but **25–100× too conservative** at 2–10 ms deadlines. That looseness is the quantitative argument for **learning** the distribution instead of bounding it.



9.3 Extended results: best implementable controller (V_w %)

Scenario	GBM + KKT (ours)	Predictive MaxWeight	CEM policy	Markov-chain KKT	Tuned-weight KKT
S low 0.50	0.15	0.14	**0.13**	0.25	0.18
S moderate 0.65	2.15	**1.62**	1.77	2.65	2.21
S high 0.80	8.00	**6.15**	6.57	8.64	7.88
S flash crowd	11.76	**9.69**	10.20	12.44	11.49
S shift	10.04	10.25	**9.73**	10.22	9.90
G moderate 0.65	5.08	7.57	**4.58**	6.44	4.98
G high 0.80	8.01	11.36	**7.17**	8.77	7.83
M moderate 0.65	0.92	1.64	0.88	1.24	**0.85**
M high 0.80	4.94	4.79	**3.93**	5.12	4.88
M shift 0.80	5.52	4.80	**4.17**	5.39	5.31

Say this plainly, it is the honest bottom line:

- Within the original (NCC) comparison set, the KKT rule still wins **all 10 scenarios** across the three families, and it is the best controller with **nothing to tune**.
- **But two controllers that use the same forecaster beat it:** the CEM policy is best in **6/10**, predictive MaxWeight in **3/10** (all on the bursty Family S), and the KKT rule with one tuned weight in 1/10.
- A Markov chain fitted online, with **no training data**, is within 3–9 % of the learned rule.

9.4 What the CEM policy learned

Per-class quantile levels **p_LL ≈ 0.8 and p_eMBB ≈ 0.5**, with an internal LL weight of about 1, and weighted water-filling. In other words: decouple the two classes' levels instead of tying them through one λ.

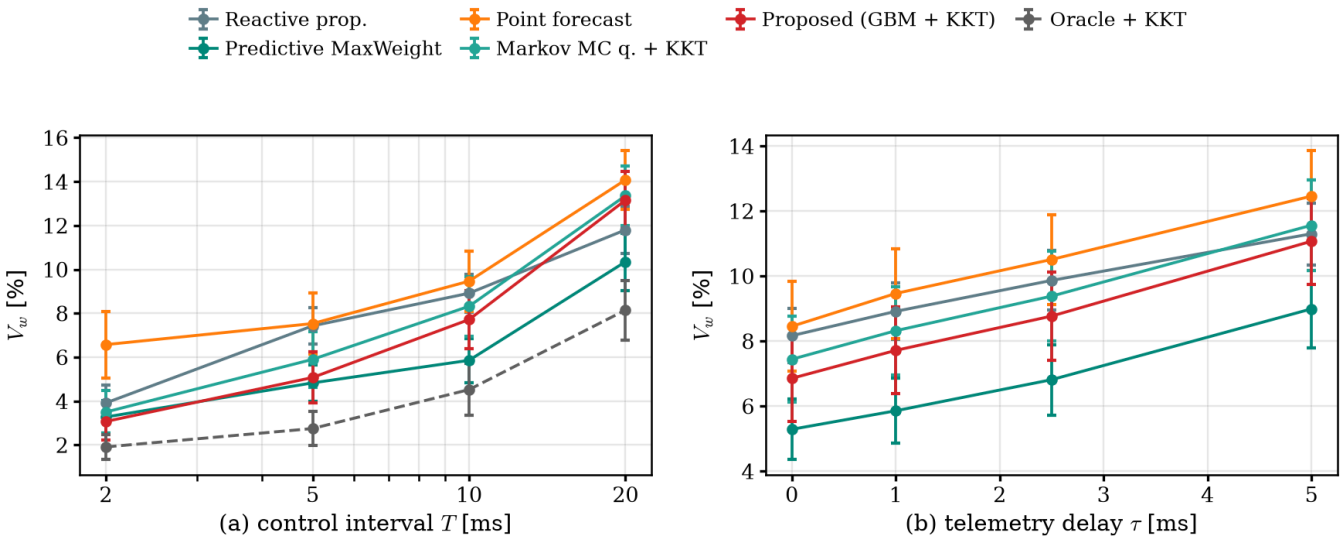
9.5 Why re-tuning the LL weight inside the KKT rule does not close the gap

- Tuned weight: 8.00 → 7.88 (S high), 8.01 → 7.83 (G high), 4.94 → 4.88 (M high). That is an order of magnitude less than the CEM gain.
- **The λ diagnostic** (high load, 3 seeds × 8,000 slots): the link is full in ~100 % of intervals (99.8 % S, 100 % G, 99.9 % M), and in **50–58 %** of them λ* ≥ w_eMBB. Then every eMBB cell already sits at its lowest quantile (recall Section 6.5); the weight has nothing left to move.
- So the gap is **structural, not a wrong constant**: CEM wins by decoupling the per-class levels and using a different sharing rule.

9.6 Operating regime (when to use which approach)

- **Short control interval (T ≤ 5 ms):** reservation (the KKT rule) wins.

- **T = 10–20 ms with bursts shorter than the interval:** back-pressure (predictive MaxWeight) wins.
- **Long bursts (FTP files, XR frames, Family G):** reservation wins at every T.



10. Limitations and what we do NOT claim

Say these **before** you are asked. Examiners respect a presenter who states the limits.

1. **No real-world validation.** Three simulated families. Family G uses standard 3GPP models, but the round-robin scheduler and the single-link fronthaul model are ours. We know of no public per-slot, per-cell PRB traces (public O-RAN datasets such as ColO-RAN are 250 ms or coarser).
2. **Simplified link:** a budget-enforced pipe; no switch queue, one traffic class per cell, no HARQ (retransmissions), downlink only. A "violation" is an O-DU queueing deadline, not end-to-end latency.
3. **No closed-loop optimality or stability claim.** The propositions cover feasibility, equal weighted risk, backlog-first and a linear miscalibration bound, for one interval.
4. **Not a universal win:** only −4 % against proportional under flash crowds; loses at $T = 20$ ms and at ≥ 0.86 load; near tie at $\tau = 5$ ms; ML adds only about 2 % under the shift; calibration hurt.
5. **Priority trade:** the claim holds for an LL weight of at least 2–3.
6. **Extended comparison:** CEM policy search and predictive MaxWeight beat the KKT rule in 9 of 10 scenarios.
7. **Decision time** is 3.5 ms in Python; a real O-DU needs a compiled version.
8. The "oracle" is a **reference, not an upper bound**.

---

11. Slide-by-slide speaking guide

These are the Part B slides of `presentation/btp_final.tex` (sections "Part B: Sharing One Fronthaul Link", "Part B: Results" and "Extended Study"). For each: what to say, and the one thing to land.

#	Slide	What to say (about 30–60 s each)	Land this
1	**Problem setting**	8 cells (3 LL, 5 eMBB) share a 25GE link, C = 12.125 Mbit per slot, aggregate peak 2.44 C on purpose. Every 10 ms a coordinator sets budgets held for the interval, on 1 ms old data. Why in advance: RIC loop ≥ 10 ms, switch shaping not per slot, DUs cannot coordinate within a slot.	"We must commit before we see the traffic."
2	**System model**	β(b) formula, β(9) = 3,387 bits, this is where Part A enters. Demand = user bits ÷ η × β. Budgets sum ≤ C, bits older than D are dropped. Metric V_w fixed before tuning.	"Load follows PRBs, not user bits."
3	**Key idea 1: r***	Smallest constant rate at which no bit misses its deadline. Long deadline → mean; short → peak burst. For 2 ms it is 2–3× the mean and skewed. need = backlog rate + r*; controllers differ only in how they estimate r*.	"A single number is a poor basis for reserving a skewed quantity."
4	**Quantile forecaster**	6 GBM quantile models, 28 past-only features, 12 s training, disjoint seeds. Calibrated within a point; 37 % error vs 52 % persistence.	"Well calibrated, cheap, CPU-only."
5	**Key idea 2: KKT rule**	Newsvendor objective; concave; KKT closed form $r_i = B_i + F^{-1}(1 - \lambda/w_i)$; one λ by bisection.	"Every cell cut at the same weighted risk; margins derived, not tuned."
6	**Properties**	Feasibility, equal weighted risk, backlog first, linear miscalibration bound $2\varepsilon w_{max} C$.	"Properties of one interval, not a closed-loop optimality claim."
7	**Control loop**	Walk the diagram left to right. Mention the edge cases and that ACI is an ablation.	
8	**Reference controllers**	Walk the table; stress the point-forecast ablation, the fixed-quantile test and two oracles. All tuned on validation; margin 0 best for all.	"The baselines were not handicapped."
9	**Evaluation protocol**	Family S (bursty, Pareto, held-out shift) and Family G (pure 3GPP). 20 seeds, identical traces, paired bootstrap and Holm-Wilcoxon.	"Fair by construction."
10	**Main result, Family S**	Best implementable in all 5. Std is shared burst timing; read the paired table next.	"Best in all five scenarios."
11	**Paired reduction**	−11 to −43 % vs point forecast; every seed against 4 baselines. Weak: −4 % vs proportional under flash crowds.	"Every seed, $p < 0.01$; and here is where it is weak."
12	**Where the gain comes from**	Unaware controllers starve LL; aware ones pay with eMBB; ours pays least eMBB with the highest utilisation; global quantile wastes 33 %.	"Margin only where the forecast is wide."
13	**Rule in action**	Reactive is one interval late at every burst; ours raises budgets at burst onset.	
14	**Ablations**	Range matters; per-cell level matters; ML useful, not necessary; ACI hurt and why.	"Most of the gain is the rule, not the model."
15	**Family G**	Reproduces on traffic we did not design; also best unweighted; transfer still beats the reactive controllers.	"Not just our own traffic."
16	**Oracle + KKT**	24–40 % of the remaining loss at high load is forecast error; at 2 ms the rule with perfect info is 1.91 % vs 6.89 % for water-fill.	"Oracle is a reference, not a bound."

#	Slide	What to say (about 30–60 s each)	Land this
17	**Priority trade caveat**	Unweighted: proportional wins in S. Weight sweep: tie at 1, ahead from 3. G at w = 1: worse.	"For operators who value an LL bit 2–3× or more."
18	**Sensitivity T and τ**	Short T: beats even the oracle water-fill. T = 20 ms or τ = 5 ms: advantage over proportional vanishes.	"Intervals up to a few times the shortest deadline."
19	**Miscalibration**	Under-reserving almost free; over-reserving degrades gradually; never worse than the point forecast.	"Degrades gracefully."
20	**Part A × Part B**	Same traffic, BFP 6/9/12/14 → load 0.44/0.65/0.86/0.99. Compression sets the regime; allocation matters at 0.5–0.8.	"Compression sets the regime, allocation decides the loss."
21	**Decision overhead**	3.5 ms in Python; the rule itself < 0.1 ms; the cost is trees; compile it or use the window fallback.	
22	**POMDP**	The real problem is intractable; three approximation families, all implemented; new Family M.	
23	**Markov theory**	r* linked to effective bandwidth; sandwich never violated; bound 25–100× loose → learn instead of bound.	"Theory explains it; learning is still needed."
24	**Three families, nineteen controllers**	Overview figure; just orient the audience.	
25	**Extended results**	Still wins the NCC set in all 10; CEM best in 6/10, predictive MaxWeight 3/10; fitted Markov chain within 3–9 %.	"Stronger controllers exist, and we found them ourselves."
26	**Why retuning does not help**	λ* ≥ w_eMBB in half of the intervals → eMBB already at its lowest quantile. Gap is structural. Regime: short T → reservation, long T with short bursts → back-pressure.	"Structure, not the constant."
27	**What we do not claim**	Read the list calmly.	
28	**Conclusions / future work**	Part B block plus the take-away: keep the structure, learn only what is uncertain. Future: real traces, a controller that switches between reservation and back-pressure, coupled calibration, compiled implementation, compression width as a per-cell knob.	

Suggested 60-second opening for Part B:

> "Part B is about sharing one fronthaul link between several cells. Operators deliberately buy a link smaller than the cells' total peak, because cells rarely burst together; in our setup the peak is 2.4 times the link. When bursts coincide, someone must lose data, and the controller has to decide in advance, every 10 milliseconds, on telemetry that is already a millisecond old, while low-latency cells can only wait 2 milliseconds. Our idea has three parts: forecast each cell's deadline-feasible rate rather than its traffic volume, forecast it as a range rather than a single number, and split the link with a closed-form rule that cuts every cell at the same priority-weighted risk. On twenty test seeds and two independent traffic families it is the best of the controllers we compared in every scenario. I will also show where it does not win, and two stronger controllers we found in the extended study."

Suggested 30-second close:

> "Forecasting the deadline-feasible rate as a range and splitting the link with a KKT rule cut weighted violations by 11 to 43 percent against the same forecaster's median, on every seed. Most of that gain comes from the rule, not the machine learning. Compression sets the load regime; allocation matters most between about 0.5 and 0.8 load. And a reward-learned policy with decoupled class levels goes further, which tells us the next step: keep the structure, learn only what is uncertain."

---

12. Likely questions and honest answers

The problem

- **Q: What is the problem in one sentence?** Several cells share a fronthaul link smaller than their total peak; decide every 10 ms how much each may send, before knowing the next 10 ms of traffic.
- **Q: Why not decide every slot?** The coordination loop (near-RT RIC over E2) is ≥ 10 ms, switch shaping is not reconfigured per slot, and DU schedulers cannot consult each other within a slot. The interval sweep shows what happens from 2 ms to 20 ms.
- **Q: Could the DU just export its schedule?** Yes, where such an interface exists (e.g. CTI for PON fronthaul); then you do not need to forecast. Our setting is when budgets must be committed before the schedule is known.
- **Q: Why is fronthaul load not just user traffic?** 7-2x carries IQ per scheduled PRB; load = user bits ÷ spectral efficiency × 3,387 bits. Same user throughput can mean 2–3× different fronthaul load.
- **Q: What exactly is a violation?** Bits that wait in the DU queue longer than their class deadline (2 ms or 10 ms) are dropped and counted. It is a DU queueing deadline, not end-to-end latency.

The method

- **Q: What is new?** None of the ingredients alone (quantile boosting, constrained newsvendor/KKT, water-filling, conformal inference, network-calculus rates). The contribution is the **combination and its fair evaluation** in shared fronthaul: forecast r* not volume, as a range, and cut every cell at the same weighted risk under the link constraint. Say "proposed method" or "candidate contribution", not "novel algorithm".
- **Q: Closest prior work?** Lagén et al. 2022 (shared 7-2x link, reactive per-slot compression control, no forecast). Bega et al. DeepCog/AZTEC (forecast-driven slicing, single number, no per-cell deadlines). Cohen et al. 2023 (conformal prediction for one URLLC stream; our ablation shows per-stream correction does not carry over to coupled cells).
- **Q: Why forecast r* and not traffic?** Because the deadline is what matters; for a 2 ms class r* is the peak burst rate, which an average hides.
- **Q: Why a range and not one number?** r* is skewed; one number under-serves bursts and a fixed margin wastes capacity (every positive margin was worse on validation).
- **Q: Is the allocation optimal?** Only for the one-interval surrogate (separable and concave, so KKT gives its global optimum). No closed-loop claim.
- **Q: Why gradient boosting and not an LSTM or a transformer?** Six models train in 12 s on a CPU, coverage within a point of nominal, and the ablation shows the model is not where most of the gain is. A sequence model is a possible improvement.
- **Q: Why ML at all?** Useful, not necessary. The window-quantile version already beats every baseline; ML adds 7–37 % on normal traffic, about 2 % under shift.
- **Q: Why did calibration (ACI) hurt?** LL misses in a coupled split are partly caused by the shortage split, not by forecast bias, so raising LL levels over-reserves LL and starves eMBB. Coupled calibration is future work.

The evaluation

- **Q: Any train/test leakage?** No. Disjoint seed ranges, forecaster frozen before testing, features from past slots only (unit-tested), normalisation by configuration constants, tuning on validation only, the shift scenario never seen in training or tuning.
- **Q: Why synthetic traffic?** No public per-cell, per-slot PRB fronthaul traces. That is why Family G is built purely from 3GPP models, and Family M from Markov models.
- **Q: Why are violation ratios so high (8–15 % at 0.8 load)?** Deliberately heavy-tailed bursts at 80 % mean load with budgets held 10 ms; even the oracle loses 13 % of eMBB bits. The paired differences are the result.
- **Q: Is the oracle an upper bound?** No. It is a perfect single-number forecast fed to water-fill. Our rule beats it at short intervals.
- **Q: Isn't the weighted metric chosen to favour you?** Fair question. We show the unweighted ratio and a weight sweep: proportional wins unweighted in Family S; our rule ties at $w = 1$ and leads from $w = 3$. The claim is scoped to operators who weight LL 2–3× or more.
- **Q: How were the baselines tuned?** Grid on validation seeds (margins, windows). Margin 0 was best for all; all redistribute unused capacity.
- **Q: Fairness and utilisation?** Jain's index > 0.98 for everyone; ours has the highest utilisation within the deadline-aware family (72 % vs 67–70 %).

Failures and future

- **Q: Where does it fail?** Flash crowds (only 4 % vs proportional), $T = 20$ ms (loses), $\tau = 5$ ms (near tie), ≥ 0.86 load (loses), ML under shift (small gain), calibration (hurt), and in the extended study CEM and predictive MaxWeight beat it in 9/10 scenarios.
- **Q: If CEM is better, why present the KKT rule?** It has nothing to tune and a closed form with provable properties; CEM needs reward-driven training per setting. The extended study shows exactly where and why the stronger controllers win (decoupled class levels, back-pressure at long intervals with short bursts), which is the path forward.
- **Q: How expensive is it?** 3.5 ms per 10 ms interval in Python; the rule is < 0.1 ms; the cost is 1,800 trees. Compile it, shrink the model, or use the window fallback.
- **Q: How do Parts A and B connect?** β, the cost of one PRB-layer, is the compression setting. Compression moves the system along the load axis; the allocation decides the loss at that load.
- **Q: What is next?** Measured PRB traces; a controller that switches between reservation and back-pressure based on burst length vs interval; coupled calibration; a compiled O-DU implementation; the compression width as a per-cell knob inside the allocation rule.
- **Q: Where is this being published?** The conference version targets NCC 2027 (IISc Bengaluru, Networks track; 6 pages, double-blind, so the submitted PDF is anonymised). An 11-page extended manuscript is in `paper/extended/`.

---

13. Cheat sheet: the numbers to remember

Setup * 8 cells: 3 LL (2 ms, weight 3) + 5 eMBB (10 ms, weight 1) * Link 25 Gbit/s; 3 % reserved; C = 12.125 Mbit per 0.5 ms slot * β(BFP-9) = 3,387 bits per PRB-layer per slot; cell peak 7.4 Gbit/s * Aggregate peak = 2.44 × C * Control interval T = 10 ms (20 slots); telemetry delay τ = 1 ms (2 slots)

Forecaster * 6 quantiles (5/25/50/75/90/95 %), 300 trees, 15 leaves, 28 features * 101k rows, 12 s training * Coverage 5.6 / 25.6 / 50.7 / 75.3 / 89.9 / 94.9 % * Median error 37 % (vs 52 % persistence)

Main results (Family S, 20 seeds) * Proposed V_w: 0.15 / 2.15 / 8.00 / 11.76 / 10.04 % (low / moderate / high / flash / shift) * vs point forecast: −11 to −43 %, 20/20 seeds * vs fixed global quantile: −22 to −54 % * vs deadline-aware reactive: −19 to −70 % * vs MaxWeight: −18 to −83 % * vs reactive proportional: −4 % (flash) to −81 % (low) * ML adds 7–37 % (2 % under shift) * At high load: LL 3.4 %, eMBB 12.3 %, utilisation 72.2 %

Family G (3GPP), high load * Proposed 8.01 %: −11 % vs point, −31 % vs deadline-aware, −50 % vs proportional; 20/20

Forecast error share (oracle + KKT gap): 24–40 % at high load

Regime * T: wins at 2/5/10 ms; loses at 20 ms * τ: margin over proportional 16 % → 2 % at 5 ms * Load: allocation matters at 0.5–0.8; BFP 6/9/12/14 → 0.44/0.65/0.86/0.99 * Weight: needs $w_{LL} \geq 2$–3

Extended study * CEM best in 6/10, predictive MaxWeight 3/10, tuned-weight KKT 1/10 * Bound 25–100× conservative; $\lambda^* \geq w_{eMBB}$ in 50–58 % of intervals * Decision time 3.5 ms (Python), rule < 0.1 ms

---

14. Glossary

- **O-RU / O-DU:** radio unit on the tower / processing unit nearby.
- **Fronthaul:** the link between them. **7-2x:** the O-RAN split where the O-RU sends IQ samples per scheduled PRB.
- **IQ samples:** complex radio samples (in-phase and quadrature).
- **BFP:** block floating point compression: shared exponent, short mantissa.
- **PRB:** physical resource block, 12 subcarriers × 1 slot; 273 per 100 MHz cell.
- **Layer / PRB-layer:** one MIMO stream / one PRB on one layer.
- **Spectral efficiency (η):** user bits per resource; depends on radio quality.
- **eCPRI:** protocol carrying IQ over Ethernet.
- **Near-RT RIC / E2:** O-RAN's near-real-time controller and its interface to the base station, running at 10 ms to 1 s.
- **Statistical multiplexing:** buying a link smaller than the total peak because peaks rarely coincide.
- **eMBB / LL:** broadband traffic (10 ms deadline) / low-latency traffic (2 ms deadline).
- **Deadline violation:** a bit dropped because it waited longer than its deadline.
- **V_w:** priority-weighted violation ratio, the main score.
- **r* (deadline-feasible rate):** smallest constant rate that serves the coming arrivals with no bit late.
- **Backlog rate B_i:** the rate needed to clear what is already queued before it expires.
- **Quantile:** the value below which a given percentage of outcomes fall.
- **Pinball loss:** the loss function used to train quantile models.
- **Gradient boosting (GBM):** an ensemble of small trees, each correcting the last.
- **Calibration / coverage:** how often reality actually falls below each predicted quantile.
- **Newsvendor problem:** how much to stock under uncertain demand.
- **KKT conditions:** optimality conditions of a constrained optimisation problem.
- **λ (Lagrange multiplier):** the "price" of link capacity; one number for all cells.
- **Bisection:** halve an interval repeatedly to find the λ that makes budgets sum to C.
- **Water-fill:** fill needs in order (or proportionally) until the link is full.
- **MaxWeight / back-pressure:** serve the queues with the largest weight × length first.
- **Predictive MaxWeight:** MaxWeight with a look-ahead from the forecast.
- **ACI:** adaptive conformal inference: an online rule that widens or narrows forecast ranges after misses.
- **CEM:** cross-entropy method, a simple policy search from reward.
- **POMDP:** partially observed Markov decision process.
- **Effective bandwidth:** classical queueing quantity between the mean and peak rate of a source.
- **Paired comparison:** compare two controllers on the same seed, then aggregate the differences.
- **Bootstrap CI:** confidence interval obtained by resampling.
- **Wilcoxon signed-rank test:** a non-parametric test for paired data.
- **Holm correction:** adjusts p-values when many comparisons are made.
- **Jain's fairness index:** 1 means perfectly equal service ratios.

15. Where things live in the repository, and a live demo

What	Where
Simulator, traffic, controllers, forecaster, KKT rule	`fhlm/` (`simulator.py`, `traffic*.py`, `controllers*.py`, `forecast.py`, `proposed.py`, `demand.py`)
Theory (extended)	`fhlm/theory.py`, `fhlm/theory_validation.py`, `fhlm/markov.py`
Tests	`tests/test_fhlm.py`, `tests/test_fhlm_extended.py`
All results (CSV, logs, figures)	`results/fhlm/`, figures in `results/fhlm/figures/`
Method with proofs	`docs/fhlm/method.md`
Results discussion	`docs/fhlm/results_summary.md`, `docs/fhlm/extended_study.md`
Literature review	`docs/fhlm/literature_review.md`
Reviewer-style critique and responses	`docs/fhlm/review_and_response.md`
Conference paper (NCC 2027, anonymised)	`paper/main.tex`, `paper/main.pdf`
Extended manuscript	`paper/extended/main.tex`, `paper/extended/main.pdf`
Final slides	`presentation/btp_final.tex`, `presentation/btp_final.pdf`

Every number in the papers is a LaTeX macro generated from the CSVs (`fhlm/make_paper_tables.py`, `fhlm/make_extended_tables.py`), so text and data cannot drift apart. That is a good point to make if asked about reproducibility.

Live demo (about 3 minutes, CPU only):

```
pip install -r requirements.txt
python -m pytest tests/test_fhlm.py -q                  # link limit every slot, no future leak
python -m fhlm.run_experiments demo --scenario high     # one trace, 3 controllers, ~10 s
python -m fhlm.make_figures                             # rebuild figures from saved CSVs
```

Expected order in the demo output: proposed < point forecast < deadline-aware reactive. Then open `results/fhlm/figures/fig6_demo_timeseries.png` and explain it as in Section 8.4.

Full reproduction (if asked): `python -m fhlm.train`, then `python -m fhlm.run_experiments tune`, `main` (~12 min for 1,000 runs on 4 cores), `sweep`, `compression --scenario moderate`, and `bash experiments/run_extended.sh` for the extended study (about 2 hours).